

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 2
Nom, prénom : BOUKHATEM Mohamed		N° candidat : 02248923847
Épreuve ponctuelle ☒ Contrôle en cours de formation ☐		Date : 22 / 03 / 2026
Organisation support de la réalisation professionnelle ObsiLock est un coffre-fort numérique client/serveur développé dans le cadre du BTS SIO option SLAM. L'objectif est de permettre à des utilisateurs de stocker, organiser et partager des fichiers de manière sécurisée, avec chiffrement systématique au repos côté serveur.		
Intitulé de la réalisation professionnelle ObsiLock – Coffre-Fort Numérique		
Période de réalisation : 10/02/2026 – 22/03/2026 Lieu : Nice Modalité : ☐ Seul(e) ☒ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Ressources fournies : Cahier des charges du projet ObsiLock, contrat d'API au format OpenAPI v1 (openapi.yaml) défini en équipe, accès au serveur mutualisé distant (iris.a3n.fr), documentation du micro-framework Slim 4 (PHP) et de la librairie Medoo, configuration Docker préétablie (Dockerfile, docker-compose.yml, init.sql). Résultats attendus : Architecture Client/Serveur fonctionnelle et déployée (JavaFX + API PHP REST), stockage et accès aux fichiers avec chiffrement au repos (LibSodium), authentification sécurisée via JWT, gestion des fichiers (upload, download, versioning,), partage sécurisé par liens, gestion des quotas, interface JavaFX avec thème dynamique, documentation technique et utilisateur.		
Description des ressources documentaires, matérielles et logicielles utilisées² Langages et frameworks : PHP 8 + Slim Framework 4 (backend REST), Java 17 + JavaFX 17 (client desktop), FXML, CSS. Bibliothèques : LibSodium (chiffrement XSalsa20-Poly1305), Firebase PHP-JWT, Medoo ORM, Jackson (parsing JSON). Infrastructure : Docker, MySQL 8.0, Traefik (reverse proxy HTTPS), serveur VPS distant. Outils : Git / GitHub, IntelliJ IDEA + Scene Builder, Postman (tests API), ANTIGRAVITY (IDE).		
Modalités d'accès aux productions³ et à leur documentation⁴: Code source Backend et Client JavaFX disponibles sur le dépôt Git de l'établissement (dossier coffreFortJava-main/). API accessible en production : https://api.obsilock.iris.a3n.fr:4433 Documentation technique et utilisateur en Markdown dans /DOCS/. Spécification OpenAPI consultable via openapi.yaml (Swagger Editor). >>> Lien vers la page du portfolio présentant cette réalisation professionnelle		

¹ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

² Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

³ Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁴ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

Épreuve E5 - Conception et développement d'applications (option SLAM)

ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Contexte

ObsiLock est un coffre-fort numérique client/serveur développé dans le cadre du BTS SIO. L'objectif est de permettre à des utilisateurs de stocker, organiser et partager des fichiers de manière sécurisée, avec chiffrement systématique au repos côté serveur.

Architecture technique

L'application repose sur une architecture Client/Serveur découplée et stateless :

- Backend : API REST en PHP 8 + Slim Framework 4, conteneurisée via Docker (PHP/Apache + MariaDB + PhpMyAdmin), déployée derrière Traefik (reverse proxy HTTPS) sur un VPS distant.
- Frontend : Application Desktop Java 17 + JavaFX 17, communiquant avec l'API via HTTP REST avec authentification Bearer JWT.
- Base de données : MySQL 8.0 avec 8 tables (users, folders, files, file_versions, shares, downloads_log, settings).

Missions réalisées

Mission 1 – Authentification sécurisée (JWT)

Connexion via POST /auth/login avec vérification bcrypt et génération d'un JWT signé (HS256, expiry 1h). Toutes les routes protégées valident le token via un middleware Slim.

Mission 2 – Gestion des fichiers avec chiffrement

Upload avec chiffrement en streaming (LibSodium XSalsa20-Poly1305, blocs 8 Ko). Téléchargement avec déchiffrement à la volée. Versioning immuable (numéro de version, checksum SHA-256, clé de chiffrement propre à chaque version).

Mission 3 – Partage et quotas

Partage par lien sécurisé (token opaque, date d'expiration, quota d'usages, révocation). Journal de téléchargements (downloads_log). Gestion des quotas de stockage par utilisateur (50 Mo par défaut) avec barre de progression.

Mission 4 – Interface JavaFX et sécurité HTTP

Interface Desktop avec thème dynamique Obsidian (dark) / Emerald Green (clair). Middleware de sécurité HTTP (X-Frame-Options, CSP, HSTS) et rate limiting par IP sur les routes sensibles.

Mission 5 – Documentation et tests

Collection Postman (scénarios fil rouge + cas d'erreurs). Documentation technique et utilisateur en Markdown (/DOCS/ : MCD, MLD, MPD, diagrammes UML, GANTT, guide utilisateur).

Contraintes

Respect du contrat d'API OpenAPI v1 défini en équipe le premier jour. Architecture stateless obligatoire. Fichiers jamais stockés en clair sur le serveur. Déploiement en production sur serveur distant via Docker.

Bilan

L'application est fonctionnelle et déployée en production. L'ensemble des fonctionnalités (authentification, gestion de fichiers chiffrés, versioning, partage, quotas, interface graphique) ont été livrées. La documentation technique et utilisateur est disponible dans le dépôt Git